# สรุปงาน Event tree และ Guard tree

แบบจำลอง 198 ตัว · ตรวจกับแผนภาพต้นฉบับ 197 ตัว และกับไฟล์ Rodin 1 ตัว (model_001)

| 198 | 198 / 198 | 334 | 3,244 |
|---|---|---|---|
| แบบจำลอง | ผ่านการตรวจครบทุกข้อ | รหัส guard | รหัส event |

## 1. ภาพรวม

สร้าง tree 2 แบบให้แบบจำลอง Event-B ทุกตัวที่ได้จาก ChronoSeq-B โดยใช้ guard ในแบบจำลองตามเดิม ไม่แก้ไข

| tree | node | สร้างจาก |
|---|---|---|
| Event tree | event ของ Event-B (send และ receive แยกกัน) | guard ลำดับข้อความ เช่น `p ∈ receivedMessages` |
| Guard tree | เงื่อนไขจากแผนภาพ (loop, alt, opt) | guard เงื่อนไข เช่น `mode = GOLD` |

## 2. การตั้งรหัส

| | ตัวอักษร | ตัวเลข | ตัวอย่าง |
|---|---|---|---|
| guard | จุดตัดสินใจ (เงื่อนไขที่นิพจน์ด้านซ้ายเหมือนกัน ใช้ตัวอักษรเดียวกัน) | ทางเลือกของจุดตัดสินใจ | `mode = GOLD` → b1<br>`mode = SILVER` → b2 |
| event | ลำดับขั้น (event ที่ต่อกันเป็นเส้นตรงได้ตัวอักษรใหม่ทุกขั้น) | ทางเลือกที่จุดแยก (alt หรือท้าย loop) | event แรกของ 3 กิ่ง → m1, m2, m3 |

event ที่กิ่งมารวมกันได้ตัวอักษรใหม่ ถ้าทางเลือกใดเป็นจุดรวม ทางเลือกอื่นที่จุดแยกเดียวกันใช้ตัวอักษรนั้นด้วย รหัสจึงเหมือนเดิมไม่ว่าจะไล่กิ่งใดก่อน

### ตัวอย่าง: model_141 PharmacyDelivery (opt ตามด้วย alt 3 ทาง)

Guard tree

```
(root)
└── a1  needAck = 1
    ├── b1  mode = GOLD
    ├── b2  mode = SILVER
    └── b3  mode = BRONZE
```

Event tree

```
(root)
└── a1 → b1 → c1 → d1 → e1 → f1 → g1 → h1 →
      i1 → j1 → k1 → l1
    ├── m1 → n1 → o1 → p1 → q1 → r1
    ├── m2 → n2 → o2 → p2 → q1 → r1
    └── m3 → n3 → q1 → r1
```

tree ทั้งสองแบบเก็บแยกไฟล์และแยกเอกสาร รหัส guard และรหัส event เป็นคนละชุด ไฟล์ `guard_tree.txt` และ `event_tree.txt` มีเฉพาะเส้นทาง บรรทัดละ 1 เส้นทาง เช่น `a1,b1` ส่วนความหมายของรหัสอยู่ในไฟล์ `guard_codes.txt` และ `event_codes.txt`

## 3. การตรวจสอบความถูกต้อง

เขียนโปรแกรมตรวจแยกต่างหาก (`tools/verify_trees.py`) ให้อ่านแผนภาพ draw.io ต้นฉบับโดยตรง ได้แก่ กรอบ alt, opt, loop, par เส้นแบ่งกิ่ง เงื่อนไข และข้อความ แล้วคำนวณ tree ที่ควรได้ เทียบกับ tree ที่สร้างไว้ 8 ข้อ โปรแกรมนี้ไม่ใช้โค้ดของสูตรแปลงหรือของตัวสร้าง tree การตรวจจึงเป็นอิสระจากงานที่ถูกตรวจ

| ข้อ | เรื่อง | สิ่งที่ตรวจ | ผ่าน |
|---|---|---|---|
| C1 | ข้อความ | ทุกข้อความในแผนภาพมี event send และ receive ครบ และเรียงลำดับตรงกัน | 198 / 198 |
| C2 | guard ของ event | เงื่อนไขที่ครอบข้อความในแผนภาพตรงกับ guard ของ event send และ receive ของข้อความนั้น | 197 / 197 |
| C3 | รหัส guard | ทุกเงื่อนไขในแผนภาพมีรหัส และไม่มีรหัสเกิน | 198 / 198 |
| C4 | ลำดับ event | event ก่อนหน้าของทุก event ตรงกับแผนภาพ รวมถึงจุดแยกและจุดรวมของ alt | 197 / 197 |
| C5 | loop | ทุก loop มี event นับรอบ มี guard ถูกต้อง อยู่ท้าย loop และวนกลับไปยัง event แรกของ loop | 197 / 197 |
| C6 | รหัส event | event หนึ่งมีรหัสเดียว ทางเลือกที่จุดแยกใช้ตัวอักษรเดียวกันโดยตัวเลขไม่ซ้ำ และเส้นตรงไม่ใช้ตัวอักษรซ้ำ | 198 / 198 |
| C7 | เส้นทาง guard | เส้นทางใน guard tree ครบทุกทางที่เป็นไปได้ในแผนภาพ | 197 / 197 |
| C8 | ไฟล์ | ไฟล์ในชุดเทรน (models/, data/*_trees.jsonl, guard_codes.csv, event_codes.csv) ตรงกับต้นฉบับ | ทุกไฟล์ |

C2, C4, C5 และ C7 ต้องใช้แผนภาพ จึงตรวจ 197 ตัว ส่วน model_001 ไม่มีแผนภาพ จึงตรวจ C1 และ C3 กับไฟล์ .bpo ของ Rodin แทน ผลรายแบบจำลองอยู่ใน `check.csv`

### ทดสอบโปรแกรมตรวจ

เพื่อยืนยันว่าโปรแกรมตรวจพบข้อผิดพลาดได้จริง ได้แก้ tree ที่ถูกต้องให้ผิดโดยเจตนา 10 แบบ โปรแกรมตรวจพบ **10 จาก 10** แบบ

| ข้อผิดที่ใส่ | แบบจำลอง | ตรวจพบในข้อ |
|---|---|---|
| ลบ guard ของ event | model_109 | C2 |
| เปลี่ยนข้อความของ guard | model_109 | C2 |
| เพิ่มรหัส guard ที่ไม่มีในแผนภาพ | model_109 | C3 |
| ตัดกิ่ง alt ออก | model_109 | C4 |
| ให้ loop วนกลับผิด event | model_109 | C5 |
| ให้ทางเลือกที่จุดแยกใช้คนละตัวอักษร | model_109 | C6 |
| ลบเส้นทาง guard | model_109 | C7 |
| ตัดลำดับ event | model_109 | C4 |
| สลับลำดับในเส้นทาง guard | model_141 | C7 |
| ใช้ตัวเลขทางเลือกซ้ำ | model_141 | C6 |

## ปัญหาที่พบและการแก้ไข

| ปัญหา | สาเหตุ | การแก้ไข |
|---|---|---|
| รหัส event ที่จุดแยกเดียวกันใช้คนละตัวอักษร (model_015, model_036, model_038) | ทางเลือกหนึ่งเป็นจุดรวมของ alt ก่อนหน้า ซึ่งมีตัวอักษรของตัวเองอยู่แล้ว อีกทางเลือกคือ event นับรอบ loop จึงได้ตัวอักษรใหม่ | ให้ทางเลือกอื่นใช้ตัวอักษรของจุดรวม (k1, k2) ตรวจซ้ำแล้วผ่านทุกแบบจำลอง |
| model_001 ไม่มี tree ในรอบแรก | model.txt มีเพียงชื่อ event ไม่มี guard | ใช้ event และ guard จากไฟล์ .bpo ของ Rodin |

## 4. ข้อสังเกตเกี่ยวกับสูตรแปลง

การตรวจพบลักษณะต่อไปนี้ของแบบจำลองที่ได้จากสูตรแปลง tree แสดงตามแบบจำลองจริงโดยไม่ปรับแก้

- **par** — ข้อความใน par ถูกเรียงตามลำดับบนลงล่าง ไม่มี guard ที่ให้ทำงานพร้อมกัน event tree จึงเป็นเส้นตรง
- **opt** — ข้อความหลัง opt ต้องรอข้อความสุดท้ายใน opt จึงข้าม opt ไม่ได้ tree จึงไม่มีกิ่งที่ไม่เข้า opt
- **loop** — ข้อความหลัง loop ต้องรอข้อความสุดท้ายใน loop จึงต้องทำอย่างน้อย 1 รอบ รอบถัดไปแสดงด้วย ↺
- **model_001** — แปลงด้วยสูตรรุ่นแรก loop และ par ไม่มี guard ที่อ้างข้อความก่อนหน้า event tree จึงแยกเป็น 2 กิ่งจาก root และมี 1 event ที่ไม่มีชื่อในไฟล์ Rodin (unnamed_event_N)

## 5. สถิติ

| จำนวนเส้นทางใน guard tree | 0 | 1 | 2 | 3 | 4 | 6 |
|---|---|---|---|---|---|---|
| จำนวนแบบจำลอง | 52 | 71 | 67 | 5 | 2 | 1 |

| ต่อแบบจำลอง | ต่ำสุด | มัธยฐาน | สูงสุด | รวมทั้งชุด |
|---|---|---|---|---|
| event | 8 | 15 | 30 | 3,244 |
| เส้นทางใน event tree | 1 | 2 | 6 | 387 |
| รหัส guard | 0 | 2 | 6 | 334 |
| loop | 0 | 0 | 3 | 79 |

แบบจำลอง 52 ตัวไม่มีเงื่อนไขจากแผนภาพ จึงมีเฉพาะ event tree

## 6. ไฟล์

```
EventB_PO_Training/
├── trees/
│   ├── SUMMARY.pdf          ← เอกสารนี้
│   ├── tree_summary.csv     ← ตัวเลขสรุปรายแบบจำลอง
│   ├── check.csv            ← ผลตรวจรายแบบจำลอง
│   ├── guard/
│   │   ├── GUARD_TREES.pdf  ← หลักการ ตัวอย่าง และ guard tree ของทุกแบบจำลอง
│   │   ├── guard_codes.csv  ← รหัส guard ของทุกแบบจำลอง
│   │   └── by_topic/<หัวข้อ>/<pass|fail>/<แบบจำลอง>/  ← guard_tree.txt, guard_codes.txt
│   └── event/
│       ├── EVENT_TREES.pdf  ← หลักการ ตัวอย่าง และ event tree ของทุกแบบจำลอง
│       ├── event_codes.csv  ← รหัส event ของทุกแบบจำลอง
│       └── by_topic/<หัวข้อ>/<pass|fail>/<แบบจำลอง>/  ← event_tree.txt, event_codes.txt
├── models/<id>/   ← guard_tree.txt, guard_codes.txt, event_tree.txt, event_codes.txt, *.json
└── data/guard_trees.jsonl, data/event_trees.jsonl              ← ทุกแบบจำลอง สำหรับใช้กับโปรแกรม
```

สร้างใหม่ทั้งหมดได้ด้วยคำสั่ง `python tools/make_release.py` ซึ่งจะสร้าง tree ตรวจความถูกต้อง และทำเอกสารตามลำดับ